

IoT / EMBEDDED SYSTEMS

Daily Technical Report

Project Title	IoT Training Program
Report / Day No.	Day 4
Date	22/7/2026
Prepared By	Malak Alsharqawi
Organization	Cyber Robot

Confidential — Internal Training Documentation

Table of Contents

1. Daily Overview
 - 1.1 Summary
 - 1.2 Tasks Completed Today
2. Task 1: HW-872A (ACS712-05B) Current Sensor
 - 2.1 Objective
 - 2.2 What Is the HW-872A / ACS712 Sensor
 - 2.3 How the Sensor Works (Hall Effect)
 - 2.4 Hardware Setup & 3.3V Test Results
 - 2.5 Code / Configuration
 - 2.6 Code Breakdown
3. Task 2: SRD-05VDC Relay Module
 - 3.1 Terminal Definitions
 - 3.2 Trigger Logic: Active-LOW vs Active-HIGH
 - 3.3 Safety Behavior on Disconnect & Fail-Safe Design
 - 3.4 System Behavior Matrix
 - 3.5 Identifying Relay Type (Jumper Test)
 - 3.6 Standalone Relay Test Code
4. Task 3: Push Button & LED Diagnostic Test
 - 4.1 Test Configuration
 - 4.2 4-Pin Tactile Switch Wiring
 - 4.3 Internal Pull-Up Logic
 - 4.4 Diagnostic Code & Code Breakdown
 - 4.5 Verification Results
5. Task 4: Complete System Integration
 - 5.1 System Hardware Breakdown
 - 5.2 Pinout & Wiring
 - 5.3 Circuit Flow Diagram
 - 5.4 Control & Safety Logic
 - 5.5 Final Integrated Firmware
 - 5.6 Code Breakdown
 - 5.7 Verification & Telemetry Results
6. Key Technical Concepts
7. Schematics & Photos
8. Issues & Troubleshooting
9. Results & Outcome
10. Project Documentation & Notes

1. Daily Overview

1.1 Summary

Day 4 focused on building out the motor-protection sensing and switching stack for the ESP32-S3 platform. Work began with the HW-872A (ACS712-05B) Hall-effect current sensor, which was powered and tested directly on the 3.3V rail rather than its nominal 5.0V, requiring an empirically measured zero-current baseline instead of the datasheet value. The SRD-05VDC relay module was then characterized in detail — its Active-LOW / Active-HIGH trigger logic, its NO/NC contact behavior, and the fail-safe implications of each wiring choice during a microcontroller crash or disconnection. A 4-pin tactile push button was diagnosed next, focusing on the internal leg-pairing trap that can silently short a switch to ground if wired incorrectly. Finally, all three subsystems — current sensor, relay, and push button — were combined into a single polling-based motor protection system, which was progressively hardened into a final firmware build featuring dual-action button logic (E-Stop / Reset), a 4-LED status panel, and overcurrent auto-cutoff.

1.2 Tasks Completed Today

- HW-872A / ACS712-05B current sensor — 3.3V direct integration, baseline calibration, and live current measurement
- SRD-05VDC relay module — trigger logic, NO/NC fail-safe wiring, and standalone switching test
- 4-pin tactile push button — diagnostic wiring test with LED indicator using INPUT_PULLUP
- Integrated subsystem test — relay + current sensor + E-Stop push button on one polling loop
- Final integrated motor protection system — overcurrent trip, E-Stop/Reset dual-action button, and 4-LED status panel

2. Task 1: HW-872A (ACS712-05B) Current Sensor

2.1 Objective

Integrate the HW-872A current sensor breakout (Allegro ACS712ELCTR-05B-T) with the ESP32-S3, running it directly from the 3.3V rail, and empirically calibrate its zero-current baseline and sensitivity so that real load current can be read reliably over serial.

2.2 What Is the HW-872A / ACS712 Sensor

The HW-872A is an analog current-sensing breakout module designed to measure both DC and AC electrical currents, up to $\pm 5\text{A}$ for the 05B variant. Unlike a direct shunt resistor that measures a voltage drop in series with the power line, the ACS712 uses indirect magnetic (Hall-effect) sensing, which provides galvanic isolation up to 2.1kV RMS — keeping the high-voltage power path physically separate from the low-voltage ESP32-S3.

Specification	Value
Sensor IC	Allegro ACS712ELCTR-05B-T
Current Range	$\pm 5\text{ Amps}$
Datasheet Sensitivity	185 mV/A (0.185 V/A) at 5.0V supply
Output Type	Analog voltage (VOUT)

2.3 How the Sensor Works (Hall Effect)

1. Current Conduction — current enters and exits through the heavy internal copper trace between the two screw terminals.
2. Magnetic Field Generation — current flowing through that conductor generates a proportional magnetic field (Ampere's Law).
3. Hall Element Detection — an integrated linear Hall IC senses the magnetic field without touching the conductor.
4. Voltage Output — an internal low-noise amplifier converts the sensed field into a proportional analog voltage on the OUT pin.

2.4 Hardware Setup & 3.3V Test Results

The datasheet specifies standard 5.0V operation, which yields a 2.5V zero-current baseline. For this build, the sensor was powered directly from the ESP32-S3's 3.3V rail with no additional voltage-divider resistors.

Hardware Connections

- VCC → ESP32-S3 3.3V rail
- GND → ESP32-S3 GND pin
- OUT → ESP32-S3 GPIO 4 (ADC1_CH3)

Measured Results

Test	Observation	Conclusion
No-Load Baseline (0A)	Raw ADC averaged 1961.5 → 1.581 V	3.3V supply lowers the theoretical 1.650V midpoint to 1.581V; used as the software zero-offset
Active Small Load (5mm LED)	Pin voltage rose to 1.586–1.588 V	Current registered between 0.027 A and 0.038 A (27–38 mA), confirming milliamp-level sensitivity

2.5 Code / Configuration

main.cpp — ACS712 3.3V current-sensing sketch:

```

/*
 * HW-872A / ACS712 5A Current Sensor (3.3V Direct Mode)
 * Hardware: ESP32-S3
 * Pin: GPIO 4 (ADC1)
 */

#include <Arduino.h>

// Configuration Constants
const int SENSOR_PIN = 4;           // ESP32-S3 ADC Pin
const float SENSITIVITY = 0.185;    // 0.185 V/A (185 mV/A) for ACS712 5A
const float RESTING_OFFSET = 1.581; // Measured 0A baseline voltage at 3.3V power

void setup(){
  Serial.begin(115200);
  while (!Serial && millis() < 3000); // Wait up to 3s for native USB CDC
  analogSetPinAttenuation(SENSOR_PIN, ADC_11db); // 0 - ~3.1V ADC range
  Serial.println("\n--- ACS712 3.3V Current Sensor Initialized ---");
}

```

```

}

void loop(){
  // 1. Average 50 readings to filter out analog ADC noise
  long rawSum = 0;
  for (int i = 0; i < 50; i++) { rawSum += analogRead(SENSOR_PIN); delay(2); }
  float avgADC = rawSum / 50.0;

  // 2. Convert 12-bit ADC reading (0 - 4095) to Voltage (0 - 3.3V)
  float pinVoltage = (avgADC / 4095.0) * 3.3;

  // 3. Calculate current: I = (V_measured - V_resting) / Sensitivity
  float current = (pinVoltage - RESTING_OFFSET) / SENSITIVITY;

  // 4. Output values
  Serial.print("Raw ADC: "); Serial.print(avgADC, 1);
  Serial.print(" | Pin Voltage: "); Serial.print(pinVoltage, 3);
  Serial.print(" V | Measured Current: "); Serial.print(current, 3);
  Serial.println(" A");
  delay(500);
}

```

2.6 Code Breakdown

- `analogSetPinAttenuation(SENSOR_PIN, ADC_11db)` — expands the ESP32-S3's full-scale ADC range on GPIO 4 to roughly 3.1V across the 12-bit (0–4095) resolution.
- 50-sample averaging loop — Hall-effect sensors pick up electromagnetic ripple and ADC thermal noise; averaging 50 consecutive reads smooths this into a stable measurement.
- `Pin Voltage = (Average ADC / 4095.0) × 3.3` — converts the raw digital integer into an actual physical voltage (0.000–3.300V).
- `Current (A) = (V_measured - 1.581) / 0.185 V/A` — subtracts the empirically calibrated resting offset to center zero amps, then divides by the sensitivity factor to yield net Amperes.

3. Task 2: SRD-05VDC Relay Module

A relay is an electromechanical switch that lets a low-power microcontroller signal (3.3V/5V from an ESP32) control a high-power AC or DC circuit safely.

3.1 Terminal Definitions

Pin	Role
VCC (control side)	Powers the relay's internal electromagnetic coil, typically 5V DC on modules like the SRD-05VDC
GND (control side)	Common ground reference
IN (control side)	Control signal pin, connected to a microcontroller GPIO
COM (switch side)	Common — the central power input line from the external supply / load
NO (switch side)	Normally Open — disconnected from COM when the coil is de-energized; power flows only when energized
NC (switch side)	Normally Closed — connected to COM when the coil is de-energized; power flows by default

3.2 Trigger Logic: Active-LOW vs Active-HIGH

Over 90% of low-cost hobby relay modules (such as the SRD-05VDC with a PC817 optocoupler) are Active-LOW by design.

5. Internal design — the IN pin connects through an internal LED/optocoupler to VCC (+5V).
6. LOW signal (0V/Ground) — the GPIO pulls IN to 0V, creating a 5V potential difference. Current sinks from VCC into the GPIO, lighting the optocoupler LED and energizing the relay coil.
7. HIGH signal (3.3V or 5V) — little to no voltage difference across the optocoupler LED; current stops, de-energizing the coil.

Feature	Active-LOW Relays (Standard)	Active-HIGH Relays
Relay ON Signal	LOW (0V / Ground)	HIGH (3.3V or 5V)
Relay OFF Signal	HIGH (3.3V or 5V)	LOW (0V / Ground)
Primary Circuit Design	Current sinking (optocoupler / PNP)	Current sourcing (NPN / direct drive)
Market Share	~90% of hobby modules	Rare; found on selectable boards

3.3 Safety Behavior on Disconnect & Fail-Safe Design

For an Active-LOW relay to stay energized, the microcontroller pin must actively pull IN down to ground. If the microcontroller crashes, loses power, or its signal wire is disconnected, the GPIO floats to a high-impedance state, current can no longer flow through the optocoupler LED, and the relay coil immediately de-energizes.

Terminal	GPIO Cmd	Coil State	Load State	On Disconnect/Crash	Safety
NO	LOW	ENERGIZED	LOAD ON	Coil de-energizes → NO springs open → load stops	Fail-Safe
NO	HIGH	DE-ENERGIZED	LOAD OFF	Coil stays de-energized → NO stays open → load stops	Fail-Safe
NC	LOW	ENERGIZED	LOAD OFF	Coil de-energizes → NC springs closed → load runs	Unsafe
NC	HIGH	DE-ENERGIZED	LOAD ON	Coil stays de-energized → NC stays closed → load runs	Unsafe

Takeaway: Always connect high-power loads (motors, heaters, actuators) to the Normally Open (NO) terminal, so that if the controller loses power or connection, the circuit defaults to an OPEN / stopped state.

3.4 Identifying Relay Type (Jumper Test)

For an unlabeled relay module, the trigger logic can be verified without code:

8. Connect VCC to 5V and GND to ground on a bench supply.
9. Touch a jumper wire on the IN pin to GND (0V) — if the relay clicks on and its LED lights, it is Active-LOW.
10. Touch the IN pin to 5V — if the relay clicks on and its LED lights, it is Active-HIGH.

3.5 Standalone Relay Test Code

A minimal script used to verify the SRD-05VDC's switching action by toggling the relay every 2 seconds:

```
#include <Arduino.h>

const int RELAY_PIN = 5;  // GPIO 5 connected to Relay IN
```

```
// Active-LOW configuration:
#define RELAY_ON LOW
#define RELAY_OFF HIGH

void setup(){
  Serial.begin(115200);
  while (!Serial && millis() < 3000);
  pinMode(RELAY_PIN, OUTPUT);
  digitalWrite(RELAY_PIN, RELAY_OFF); // Ensure motor starts OFF
  Serial.println("\n--- Testing SRD-05VDC Relay Control ---");
}

void loop(){
  Serial.println("Relay ON -> Motor Running...");
  digitalWrite(RELAY_PIN, RELAY_ON);
  delay(2000);

  Serial.println("Relay OFF -> Motor Stopped.");
  digitalWrite(RELAY_PIN, RELAY_OFF);
  delay(2000);
}
```

Verification

- Relay switching — a clean mechanical "click" was heard every 4 seconds.
- Motor operation — the motor spun during RELAY_ON and halted completely during RELAY_OFF.

4. Task 3: Push Button & LED Diagnostic Test

Diagnostic procedure for validating a 4-pin tactile push button and its logic-state transitions on the ESP32-S3, verified with an LED indicator and INPUT_PULLUP mode, ahead of integrating the button as an E-Stop input.

4.1 Test Configuration

Component	Pin / Configuration	Electrical Role
Push Button Input	GPIO 6	INPUT_PULLUP (pushed = LOW / ground)
Diagnostic LED	GPIO 7	Digital OUTPUT (lit when button pressed)
Common Ground	GND	Shared reference ground for button & LED

4.2 4-Pin Tactile Switch Wiring

Standard 4-pin tactile push buttons contain two pairs of legs permanently connected internally in parallel: Leg A/Leg B are shorted on one side, and Leg C/Leg D are shorted on the other. When pressed, an internal metallic dome bridges the two sides.

Wiring Trap: Connecting GPIO 6 to Leg A and GND to Leg B (i.e. the same internal side) creates a permanent short circuit — GPIO 6 stays LOW regardless of whether the button is pressed. The foolproof rule is to always wire across opposite diagonal pins (e.g. GPIO 6 to Leg A, GND to Leg D), guaranteeing a Normally Open connection.

4.3 Internal Pull-Up Logic (INPUT_PULLUP)

Configuring `pinMode(BUTTON_PIN, INPUT_PULLUP)` enables the ESP32-S3's internal $\sim 45k\Omega$ pull-up resistor to the 3.3V rail, so the pin idles HIGH and reads LOW only when the button completes the circuit to ground.

Physical State	Switch Path	Voltage on GPIO 6	Digital Read	LED (GPIO 7)
Idle (not pressed)	Open circuit	3.3V (pulled HIGH)	HIGH (1)	LOW (OFF)
Pressed	Closed to GND	0V (pulled to ground)	LOW (0)	HIGH (ON)

4.4 Diagnostic Code & Code Breakdown

```
#include <Arduino.h>

const int BUTTON_PIN = 6; // GPIO 6 -> Push Button
const int LED_PIN = 7; // GPIO 7 -> Indicator LED

void setup(){
  Serial.begin(115200);
  while (!Serial && millis() < 3000);
  pinMode(BUTTON_PIN, INPUT_PULLUP); // Keeps pin at 3.3V (HIGH) when open
  pinMode(LED_PIN, OUTPUT);
  digitalWrite(LED_PIN, LOW); // Ensure LED starts OFF
  Serial.println("\n--- Push Button & LED Diagnostic Test ---");
}

void loop(){
  int buttonState = digitalRead(BUTTON_PIN);

  if (buttonState == LOW) {
    digitalWrite(LED_PIN, HIGH);
    Serial.println("Button PRESSED -> State: LOW (0) | LED ON");
  } else {
    digitalWrite(LED_PIN, LOW);
    Serial.println("Button IDLE -> State: HIGH (1) | LED OFF");
  }
  delay(200);
}
```

- `pinMode(BUTTON_PIN, INPUT_PULLUP)` — activates the internal pull-up so the pin has a stable 3.3V reference when open, eliminating floating-pin noise.
- `digitalRead(BUTTON_PIN)` — samples the digital voltage each loop (3.3V $\rightarrow$ 1, 0V $\rightarrow$ 0).
- `if (buttonState == LOW)` — evaluates whether the switch is actively closing the circuit to GND, driving the LED and the serial log accordingly.

4.5 Verification Results

- Idle state — serial console prints "Button IDLE -> State: HIGH (1) | LED OFF" when untouched.
- Active state — pressing the switch immediately turns on the LED and logs "Button PRESSED -> State: LOW (0) | LED ON".
- Hardware confirmed — the diagonal wiring layout was validated and is ready to be integrated into the E-Stop loop.

5. Task 4: Complete System Integration — ESP32-S3 + Current Sensor + Relay + Push Button + Motor

With each subsystem validated individually, the current sensor, relay, and push button were combined into a single motor-protection loop, then progressively hardened into a final firmware build with dual-action button logic and a 4-LED status panel.

5.1 System Hardware Breakdown

Component	Model / Spec	Purpose in Circuit
Microcontroller	ESP32-S3-N16R8	Central control unit (reads current, drives relay & LEDs)
Switching Element	SRD-05VDC-SL-C Relay Module	Controls high-power line to the DC motor
Current Sensor	HW-872A (ACS712ELCTR-05B-T)	Measures real-time load current in series
Input	4-pin tactile push button	Dual-action E-Stop / Reset control
Load	DC Motor	Primary device being powered & monitored
Power Source 1	USB / ESP32 5V rail	Logic power for ESP32, relay coil, sensor VCC
Power Source 2	External power supply	Main driving voltage for the DC motor

5.2 Pinout & Wiring

Low-Voltage Logic Domain (ESP32-S3 side)

Component	Component Pin	ESP32-S3 Pin	Role
HW-872A Current Sensor	OUT	GPIO 4	Analog input (ADC1_CH3, ADC_11db attenuation)
HW-872A Current Sensor	VCC / GND	3.3V / GND	Direct 3.3V logic power supply
SRD-05VDC Relay	IN	GPIO 5	Digital output (Active-LOW control logic)
SRD-05VDC Relay	VCC / GND	5V / GND	5V coil power supply
Push Button	Leg A (diagonal)	GPIO 6	Digital input with INPUT_PULLUP
Push Button	Leg D (diagonal)	GND	Direct ground reference

High-Current Power Domain (Motor Circuit)

Power flows in series from the external supply, through the relay switch, through the ACS712 sensor, and into the DC motor:

```
[ Power Supply (+) ] ---> Relay [ COM ]
Relay [ NO ] ---> HW-872A Terminal 1
HW-872A Terminal 2 ---> Motor (+)

[ Power Supply (-) ] -----> Motor (-) ---> Common GND
```

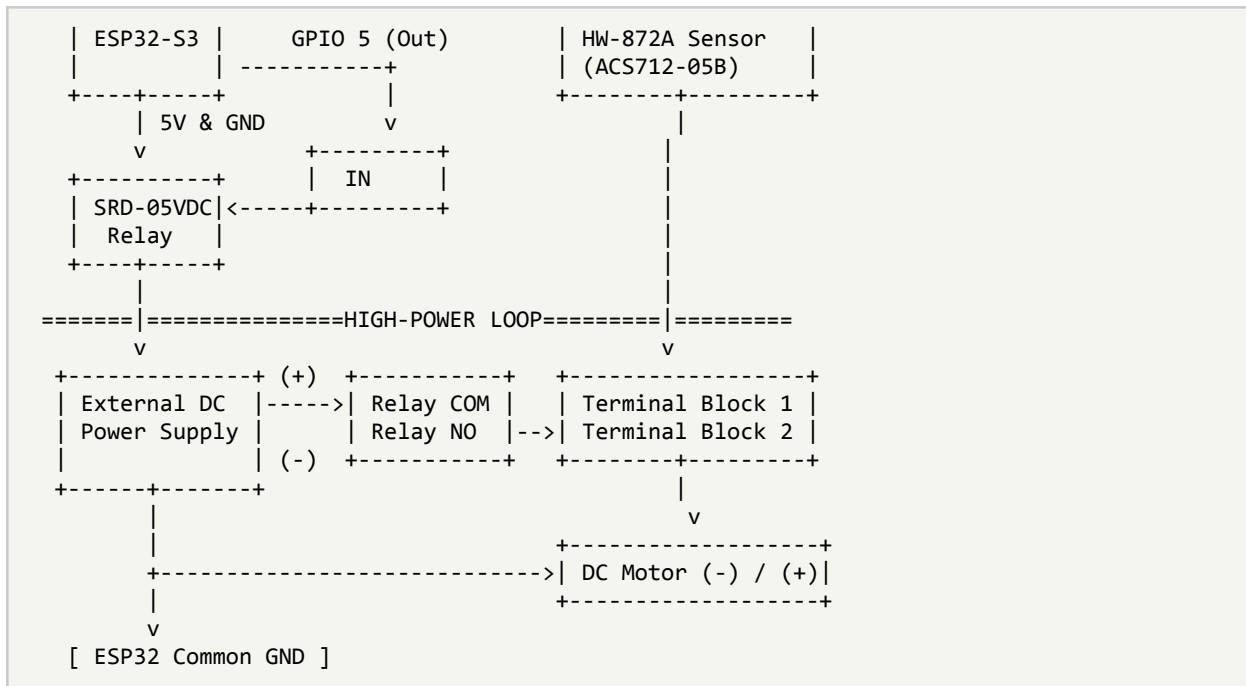
Critical: The power-supply ground, motor load ground, and ESP32-S3 GND must all be tied to a shared common ground. This common reference is required for accurate analog current measurement on GPIO 4.

5.3 Circuit Flow Diagram

```

          LOW-POWER LOGIC
        GPIO 4 (ADC)
+-----+ <-----+ +-----+

```



5.4 Control & Safety Logic

11. Normally Open (NO) vs Normally Closed (NC) — the motor power wire is connected to the NO terminal on the relay. If the ESP32 loses power, reboots, or disconnects, the relay coil de-energizes and NO springs open, immediately cutting power to the motor.
12. Active-LOW triggering — sending LOW (0V) on the relay's IN pin energizes the coil (motor ON); sending HIGH (3.3V) or disconnecting the wire de-energizes the coil (motor OFF).
13. E-Stop lockout — pressing the push button while running immediately sets an `eStopTriggered` flag, de-energizes the relay, and latches the system in a safe lockout state until reset.
14. Overcurrent auto-cutoff — the loop continuously averages 50 ADC samples from the HW-872A. If the calculated current equals or exceeds the configured limit, an `overcurrentFault` flag is set and the relay is tripped OFF instantly.

5.5 Final Integrated Firmware — Motor Protection System

The finished build adds a 4-LED status panel and turns the single push button into a dual-action control: it acts as an E-Stop while the motor is running, and as a Reset/Restart while the system is stopped or faulted.

Component / Subsystem	Pin Label	ESP32-S3 Pin	Role
HW-872A Current Sensor	OUT	GPIO 4	Analog input (ADC1_CH3, ADC_11db)
SRD-05VDC Relay	IN	GPIO 5	Digital output (Active-LOW control)
Multi-Function Button	Signal	GPIO 6	Digital input (INPUT_PULLUP, diagonal legs)
Green LED	Running	GPIO 15	Lit when motor is powered
Red LED	Stopped	GPIO 7	Lit when motor is stopped / faulted
Blue / Low LED	Normal Load	GPIO 17	Lit during normal current draw
Yellow LED	Fault / High	GPIO 16	Lit when an overcurrent trip occurs

Calibration & Theoretical Calculations

Because the sensor is powered from the 3.3V rail, both its zero-current baseline voltage and sensitivity scale proportionally relative to the nominal 5V datasheet specification:

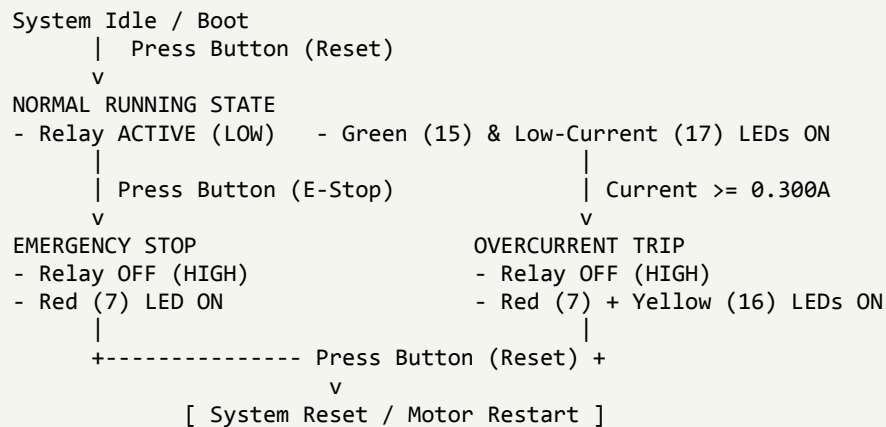
- Empirically measured 0A baseline: 1.572 V
- Scaled sensitivity: $S(3.3V) = S(5V) \times (VCC / 5.0V) = 0.185 \times (3.3/5.0) \approx 0.1221 \text{ V/A}$ (122.1 mV/A)
- Overcurrent safety threshold (trip limit): 0.300 A (300 mA)

LED Indicator Matrix & System States

System State	Relay (GPIO 5)	Green (15)	Red (7)	Low LED (17)	Yellow (16)
Initial Boot	HIGH (OFF)	OFF	ON	OFF	OFF
Normal Running ($I < 0.300A$)	LOW (ON)	ON	OFF	ON	OFF
Overcurrent Trip ($I \geq 0.300A$)	HIGH (OFF)	OFF	ON	OFF	ON
Emergency Stop	HIGH (OFF)	OFF	ON	OFF	OFF

Push Button Logic & State Transitions

The system uses falling-edge transition detection ($\text{btnState} == \text{LOW} \ \&\& \ \text{lastBtnState} == \text{HIGH}$) combined with a 50ms software debounce delay to provide dual functionality: while running, a press triggers an immediate E-Stop; while stopped or faulted, a press clears all fault flags and restarts the motor.



Complete C++ Firmware Code

```

/*
 * Integrated Motor Protection System
 * Microcontroller: ESP32-S3
 * Features: current threshold protection (0.300A), Yellow LED fault indicator,
 * dual-function push button (E-Stop when running, Reset when stopped),
 * zero-lag state changes and LED indications
 */

#include <Arduino.h>

// --- Pin Definitions ---
const int SENSOR_PIN = 4; // ACS712 Signal Output (ADC1_CH3)
const int RELAY_PIN = 5; // Relay Control Input (Active-LOW)
const int ESTOP_PIN = 6; // Multi-function Push Button (E-Stop / Reset)

```

```

const int LED_RUNNING    = 15; // Green LED: Motor Running
const int LED_STOPPED    = 7;  // Red LED: Motor Stopped / Fault
const int LED_LOW_CURR   = 17; // Normal Current Draw LED
const int LED_HIGH_CURR  = 16; // Yellow LED: High Current Fault

// --- Sensor Calibration Parameters ---
const float SENSITIVITY    = 0.1221; // Scaled for 3.3V power (122.1 mV/A)
const float RESTING_OFFSET = 1.572;  // Measured 0A baseline voltage
const float OVERCURRENT_LIMIT = 0.300; // Trip threshold (300 mA)

#define RELAY_ON  LOW
#define RELAY_OFF HIGH

bool eStopTriggered = false;
bool overcurrentFault = false;
int  lastBtnState    = HIGH;
float trippedCurrent = 0.0;

void updateLEDs(bool running, bool stopped, bool lowCurr, bool highCurr){
    digitalWrite(LED_RUNNING,  running ? HIGH : LOW);
    digitalWrite(LED_STOPPED,  stopped ? HIGH : LOW);
    digitalWrite(LED_LOW_CURR, lowCurr ? HIGH : LOW);
    digitalWrite(LED_HIGH_CURR, highCurr ? HIGH : LOW);
}

void setup(){
    Serial.begin(115200);
    while (!Serial && millis() < 3000);
    pinMode(RELAY_PIN, OUTPUT); digitalWrite(RELAY_PIN, RELAY_OFF);
    pinMode(LED_RUNNING, OUTPUT); pinMode(LED_STOPPED, OUTPUT);
    pinMode(LED_LOW_CURR, OUTPUT); pinMode(LED_HIGH_CURR, OUTPUT);
    updateLEDs(false, true, false, false); // Boot state: Stopped (Red)
    pinMode(ESTOP_PIN, INPUT_PULLUP);
    analogSetPinAttenuation(SENSOR_PIN, ADC_11db);
    Serial.println("\n=== MOTOR PROTECTION SYSTEM READY ===");
}

void loop(){
    // 1. Dual-action push button (edge-detected, debounced)
    int btnState = digitalRead(ESTOP_PIN);
    if (btnState == LOW && lastBtnState == HIGH) {
        delay(50); // debounce
        if (eStopTriggered || overcurrentFault) {
            eStopTriggered = false; overcurrentFault = false; trippedCurrent = 0.0;
            digitalWrite(RELAY_PIN, RELAY_ON);
            updateLEDs(true, false, true, false);
            Serial.println("Reset! Restarting Motor...");
        } else {
            eStopTriggered = true;
            digitalWrite(RELAY_PIN, RELAY_OFF);
            updateLEDs(false, true, false, false);
            Serial.println("Emergency Stop Activated!");
        }
    }
    lastBtnState = btnState;

    // 2. Fault lockout guards
    if (eStopTriggered) {
        digitalWrite(RELAY_PIN, RELAY_OFF); updateLEDs(false, true, false, false);
        delay(500); return;
    }
    if (overcurrentFault) {
        digitalWrite(RELAY_PIN, RELAY_OFF); updateLEDs(false, true, false, true);
    }
}

```

```

    delay(500); return;
}

// 3. Normal operation & current sensing
digitalWrite(RELAY_PIN, RELAY_ON);
long sum = 0;
for (int i = 0; i < 50; i++) {
    sum += analogRead(SENSOR_PIN); delay(2);
    if (digitalRead(ESTOP_PIN) == LOW && lastBtnState == HIGH) {
        eStopTriggered = true; digitalWrite(RELAY_PIN, RELAY_OFF);
        updateLEDs(false, true, false, false); lastBtnState = LOW; return;
    }
}
float avgADC = sum / 50.0;
float voltage = (avgADC / 4095.0) * 3.3;
float current = abs((voltage - RESTING_OFFSET) / SENSITIVITY);

// 4. Evaluate overcurrent threshold
if (current >= OVERCURRENT_LIMIT) {
    overcurrentFault = true; trippedCurrent = current;
    digitalWrite(RELAY_PIN, RELAY_OFF);
    updateLEDs(false, true, false, true);
    Serial.print("OVERCURRENT TRIP at "); Serial.println(trippedCurrent, 3);
} else {
    updateLEDs(true, false, true, false);
    Serial.print("Volts: "); Serial.print(voltage, 3);
    Serial.print(" | Current: "); Serial.print(current, 3); Serial.println(" A [OK]");
}
delay(200);
}

```

5.6 Code Breakdown

- Global parameters — SENSITIVITY (0.1221 V/A) and RESTING_OFFSET (1.572V) are the 3.3V-scaled, empirically calibrated constants; OVERCURRENT_LIMIT (0.300A) is the safety cutoff.
- updateLEDs() — a single centralized helper drives all four status LEDs together, preventing inconsistent LED combinations across state transitions.
- Falling-edge button detection — btnState == LOW && lastBtnState == HIGH registers a press exactly once, with a 50ms debounce to filter mechanical contact chatter.
- Dual action logic — the same button acts as Reset when the system is stopped/faulted, and as E-Stop when the system is normally running.
- Fault lockout guards — once eStopTriggered or overcurrentFault is set, the loop short-circuits to keep the relay OFF and the correct LEDs lit until a button reset occurs.
- Mid-loop button poll — checking the button inside the 50-sample ADC averaging loop lets the system react to an E-Stop press within the ~100ms sampling window instead of waiting for it to finish.
- current = abs((voltage - RESTING_OFFSET) / SENSITIVITY) — the abs() call ensures valid positive current magnitudes regardless of the current sensor's terminal-screw wiring direction.

5.7 Verification & Telemetry Results

Normal Operation Log

Btn State: HIGH (1) [IDLE]	Volts: 1.572 V	Current: 0.000 A
Btn State: HIGH (1) [IDLE]	Volts: 1.625 V	Current: 0.286 A

E-Stop Triggered Log

```
CRITICAL FAULT: E-Stop Activated! | Button State: LOW (0) | Relay LOCKED OFF
CRITICAL FAULT: E-Stop Activated! | Button State: LOW (0) | Relay LOCKED OFF
```

- Sensor telemetry — successfully measures load current relative to the calibrated 1.572V zero baseline.
- Relay switching — properly drives the Active-LOW coil using GPIO 5.
- Push button monitoring — directly logs live digital button states (HIGH when idle, LOW when pressed) without spurious tripping.
- Safety lockout — tripping either overcurrent or E-Stop latches the relay OFF until a software reset via the button.

6. Key Technical Concepts

6.1 Hall-Effect Current Sensing & Galvanic Isolation

The ACS712 measures current indirectly by sensing the magnetic field produced by current flowing through an internal conductor, rather than measuring a voltage drop directly on the power line (as a shunt resistor would). This provides galvanic isolation up to 2.1kV RMS between the high-voltage power path and the microcontroller's low-voltage logic.

6.2 Active-LOW vs Active-HIGH Relay Logic

Most low-cost relay modules use current-sinking optocoupler drive circuits that are Active-LOW: pulling the IN pin to ground energizes the coil, while leaving it HIGH (or floating) de-energizes it. This distinction is critical for fail-safe design, since a disconnected or crashed microcontroller will always default to the de-energized (HIGH) state.

6.3 Fail-Safe NO vs NC Wiring

Wiring a load to the relay's Normally Open (NO) contact guarantees the load loses power the instant the coil de-energizes — whether by design (turning the relay off) or by failure (power loss, crash, disconnection). Wiring to Normally Closed (NC) inverts this and is unsafe for any load that must stop on failure, such as a motor.

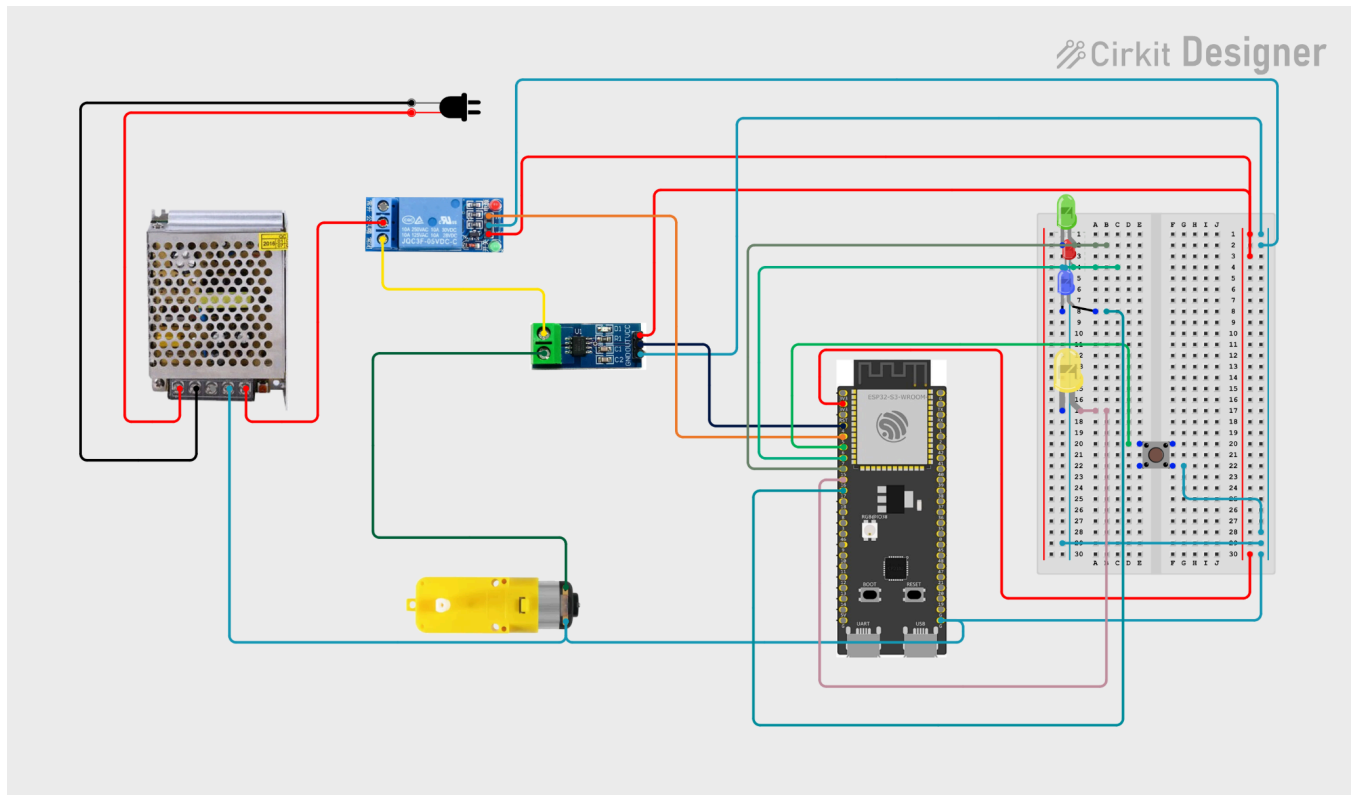
6.4 Software Debounce & Edge Detection

Mechanical push buttons produce brief electrical "chatter" as their contacts settle. Falling-edge detection (checking for a HIGH-to-LOW transition rather than just a LOW level) combined with a short delay-based debounce ensures a single physical press registers as exactly one logical event instead of several.

7. Schematics & Photos

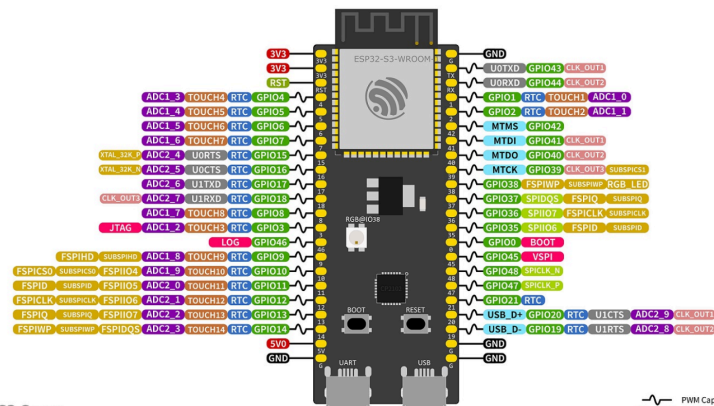
Reference pinout diagrams, component photos, and breadboard wiring captured during Day 4, alongside the component datasheets and vendor pages used to build the circuits above.

7.0 Schematic diagram for the final circuit



7.1 ESP32-S3-DevKitC-1 Pinout Reference

ESP32-S3-DevKitC-1



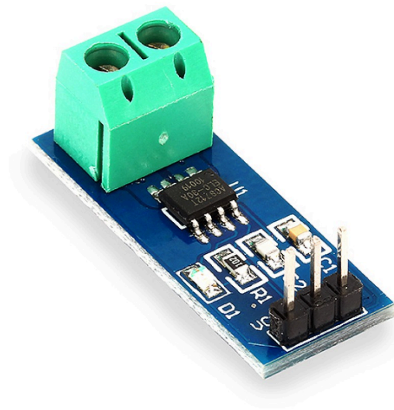
ESP32-S3 Specs
 32-bit Xtensa® dual-core @240MHz
 Wi-Fi IEEE 802.11 b/g/n 2.4GHz + BLE 5 Mesh
 512 KB SRAM (16 KB SRAM in RTC)
 384 KB ROM
 45 GPIOs, 4x SPI, 3x UART, 2x I2C,
 14x Touch, 2x I2S, RMT, LED PWM, USB-OTG,
 TWAI®, 2x 12-bit ADC, 1x LCD interface, DVP

MISC: Miscellaneous/SPI functions
 CLK_OUT: Clock Output

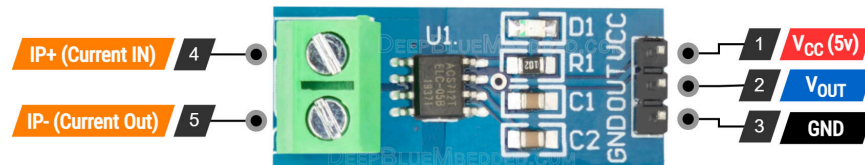
PWM Capable Pin
 GPIO Input and Output
 JTAG/USB: JTAG for Debugging and USB
 ADCX_C: Analog-to-Digital Converter
 TOUCH: Touch Sensor Input Channel
 OTHER: Other Related Functions
 SERIAL: Serial for Debug/Programming
 STRAP: Strapping Pin Functions
 RTC: RTC Power Domain (VDD3P3_RTC)
 GND: Ground
 PWD: Power Rails (3V3 and 5V)

ESP32-S3-DevKitC-1 pinout diagram, used to select GPIO 4 (ADC1_CH3), GPIO 5, GPIO 6, and the LED pins.

7.2 HW-872A / ACS712 Current Sensor



HW-872A breakout board (Allegro ACS712ELCTR-05B-T).



ACS712 AC/DC Current Sensor

Pinout

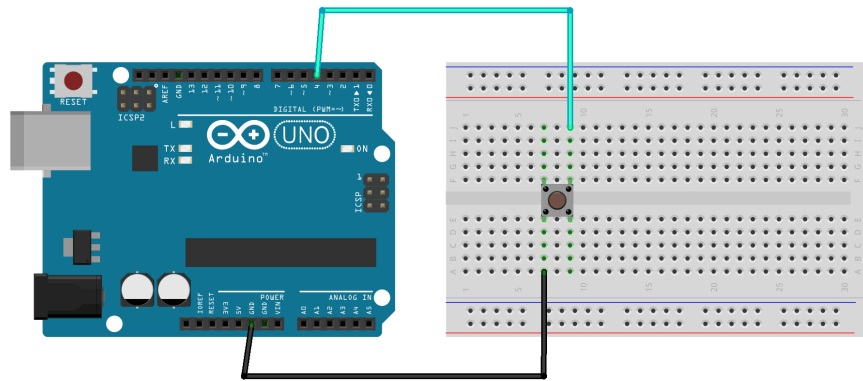
ACS712 AC/DC current sensor pinout: IP+/IP- current terminals and VCC/OUT/GND signal pins. Source: deepbluembedded.com

7.3 SRD-05VDC Relay Module



SRD-05VDC-SL-C relay module, rated 10A at 250VAC / 125VAC / 30VDC / 28VDC.

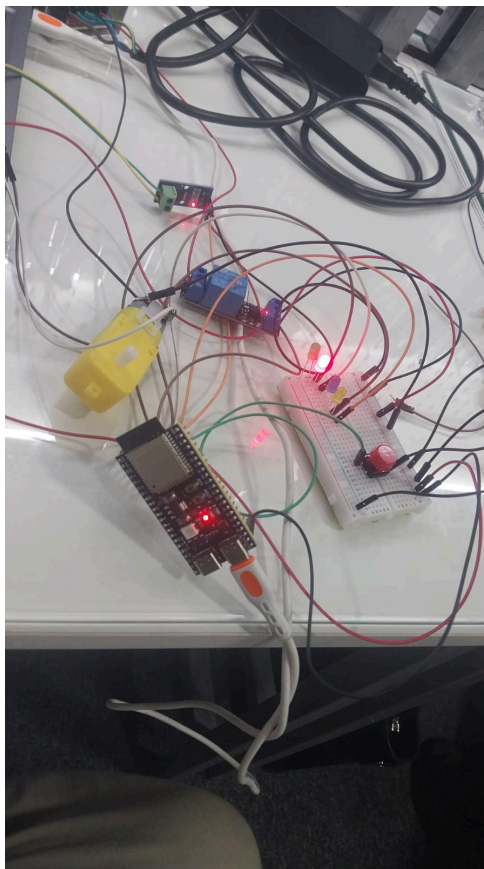
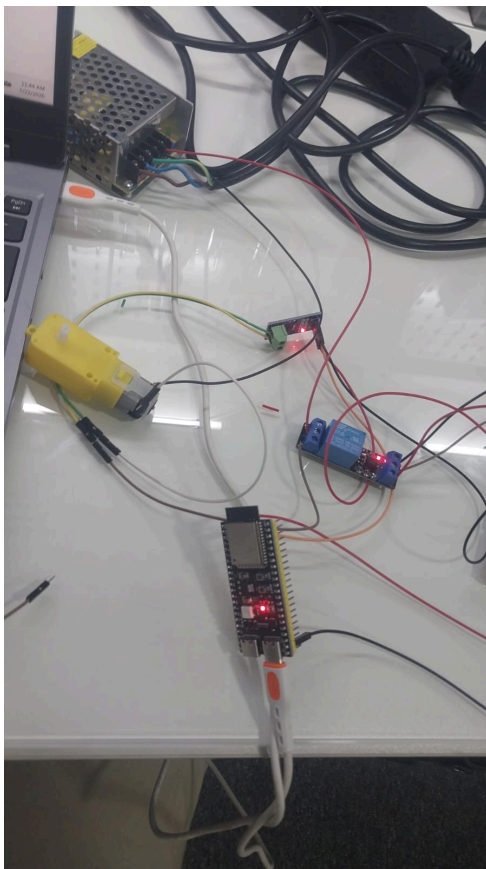
7.4 Push Button Breadboard Wiring

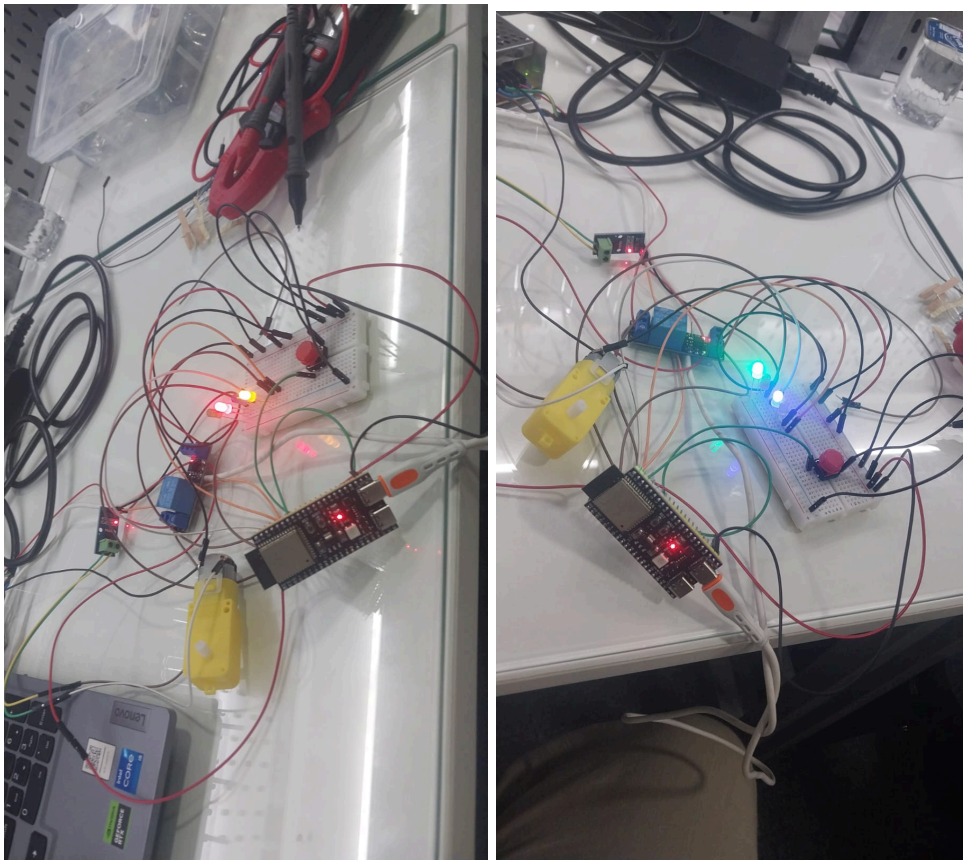


fritzing

Fritzing wiring reference for the tactile push button diagnostic test (diagonal-leg wiring to avoid an internal short).

7.5 Photos for the final circuit





8. Issues & Troubleshooting

Symptom: Relay clicking but load not switching

Early bench testing confirmed the relay coil audibly clicked on every RELAY_ON command, but this alone does not confirm the load circuit is correct — it only verifies the low-voltage control side. The high-power NO/COM wiring was independently re-checked against the fail-safe matrix in Section 3.3 to confirm the motor was wired to NO, not NC.

Symptom: Current readings drifting slightly around 0A at rest

Small positive readings (a few mA) were observed at rest due to ADC noise and the ACS712's own internal noise floor. This was mitigated by averaging 50 ADC samples per reading and, in the final firmware, by taking the absolute value of the voltage delta so wiring polarity on the current sensor's screw terminals could not produce a spurious negative-current fault.

Symptom: Push button appeared permanently pressed

When first prototyping the 4-pin tactile switch, wiring both leads to the same internally-shortened side of the button (e.g. GPIO 6 and GND both to Leg A/Leg B) created a hard short to ground, so the pin read LOW continuously regardless of the physical button state. This was resolved by always wiring across diagonal legs, as documented in Section 4.2.

9. Results & Outcome

All four subsystems were validated individually and then successfully combined into a single integrated firmware build. The current sensor accurately tracked load current relative to its calibrated 1.572V zero baseline; the relay reliably switched the motor circuit under Active-LOW control with NO-side fail-safe wiring; the push button diagnostic confirmed clean, debounced state transitions; and the final motor protection system correctly implemented a dual-action E-Stop/Reset button, a 4-LED status panel, and an automatic overcurrent trip at 0.300A — with the relay latching OFF and remaining OFF until a deliberate button reset.

10. Project Documentation & Notes

Development Notes & Key Learnings: **Day 4 Notion Notebook (shared notebook)**

- Current sensor product reference (HW-872A / ACS712): [HW-872 - 深圳市宏维微电子有限公司](#)
- ACS712 AC/DC current sensor measurement guide: [Arduino ACS712 AC/DC Current Sensor + Measurement Examples](#)
- Relay module wiring guide: [Relay Module with Arduino – Step by Step Guide - CircuitMagic](#)
- SRD-05VDC-SL-C relay datasheet: [SRD-05VDC-SL-C Datasheet - 5V, RELAY, Pinout - SONGLE](#)

End of Report